LABORATORY OBSERVATION

Course:	Full Stack Web Development
Course Code:	23CM4121
Experiment No.:	3
Student Name:	A. Bhanu Prasad
Roll No.:	A24126552068
Date:	30-08-2026

---

1. TITLE

Student Profile Manager Web Application Using JavaScript ES6 and DOM Manipulation

2. AIM

To design and develop an interactive Student Profile Manager web application using JavaScript ES6 class syntax and dynamic DOM manipulation.

3. OBJECTIVE

- Define a Student class using ES6 syntax with name, rollNumber, department, and cgpa properties.
- Capture input field data from the user and instantiate Student objects dynamically.
- Dynamically create and append structured profile cards into the DOM.
- Implement input validation and form reset after submission.

4. THEORY

Modern JavaScript (ES6+) supports object-oriented programming via the class keyword, constructor functions, and method declarations. The Document Object Model (DOM) connects web pages to scripts, allowing developers to programmatically generate HTML nodes (document.createElement) and attach them to the live page.

5. ALGORITHM / PROCEDURE

1. Create HTML structure with input fields for Name, Roll Number, Department, and CGPA.
2. Create Create Profile button and output profile container.
3. Implement Student class with constructor initializing all properties.
4. Add click event listener to Create Profile button.
5. Validate inputs; create new Student instance.
6. Generate DOM elements for student card with header, details, and delete actions.
7. Append card to profile container and reset input fields.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Student Profile Manager</title>
    <link rel="stylesheet" href="style.css">
</head>

<body>

    <div class="container">
        <h1>Student Profile Manager</h1>

        <div class="form">
            <input type="text" id="name" placeholder="Enter Name">
            <input type="text" id="rollNumber" placeholder="Enter Roll Number">
            <input type="text" id="department" placeholder="Enter Department">
            <input type="number" id="cgpa" placeholder="Enter CGPA" step="0.01">

            <button id="createProfile">Create Profile</button>
        </div>

        <div id="profileContainer"></div>
    </div>

    <script src="script.js"></script>

</body>
</html>
```

style.css

```
* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}

body {
    font-family: Arial, sans-serif;
    background: #f2f4f7;
    min-height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
}

.container {
    width: 500px;
    background: white;
    padding: 30px;
    border-radius: 12px;
    box-shadow: 0 4px 15px rgba(0, 0, 0, 0.15);
}

h1 {
    text-align: center;
    margin-bottom: 25px;
}
.form {
```

```
    display: flex;
    flex-direction: column;
    gap: 12px;
}

input {
    padding: 12px;
    border: 1px solid #ccc;
    border-radius: 6px;
    font-size: 16px;
}

button {
    padding: 12px;
    border: none;
    border-radius: 6px;
    background: #222;
    color: white;
    font-size: 16px;
    cursor: pointer;
}
button:hover {
    background: #444;
}

#profileContainer {
    margin-top: 25px;
}

.profile {
    padding: 20px;
    border: 2px solid #222;
    border-radius: 10px;
    background: #fafafa;
}

.profile h2 {
    text-align: center;
    margin-bottom: 15px;
}

.profile p {
    margin: 10px 0;
    font-size: 16px;
}
```

script.js

```
// Student class
class Student {
    constructor(name, rollNumber, department, cgpa) {
        this.name = name;
        this.rollNumber = rollNumber;
        this.department = department;
        this.cgpa = cgpa;
    }
}

// Select elements from the DOM
const nameInput = document.getElementById("name");
const rollNumberInput = document.getElementById("rollNumber");
const departmentInput = document.getElementById("department");
const cgpaInput = document.getElementById("cgpa");

const createButton = document.getElementById("createProfile");
const profileContainer = document.getElementById("profileContainer");

// Button event
createButton.addEventListener("click", function () {

    const name = nameInput.value;
    const rollNumber = rollNumberInput.value;
    const department = departmentInput.value;
    const cgpa = cgpaInput.value;

    // Check whether all fields are filled
    if (name === "" || rollNumber === "" || department === "" || cgpa === "") {
        alert("Please fill all the fields.");
        return;
    }

    // Create Student object
    const student = new Student(
        name,
        rollNumber,
        department,
        cgpa
    );

    // Clear previous profile
    profileContainer.innerHTML = "";

    // Create profile elements dynamically
    const profile = document.createElement("div");
    profile.classList.add("profile");

    const title = document.createElement("h2");
    title.textContent = "Student Profile";
    const nameElement = document.createElement("p");
    nameElement.textContent = "Name : " + student.name;

    const rollElement = document.createElement("p");
    rollElement.textContent = "Roll No : " + student.rollNumber;

    const departmentElement = document.createElement("p");
    departmentElement.textContent = "Department : " + student.department;

    const cgpaElement = document.createElement("p");
    cgpaElement.textContent = "CGPA : " + student.cgpa;

    // Add elements to profile
```

```
    profile.appendChild(title);
    profile.appendChild(nameElement);
    profile.appendChild(rollElement);
    profile.appendChild(departmentElement);
    profile.appendChild(cgpaElement);

    // Add profile to webpage
    profileContainer.appendChild(profile);
});
```

7. CODE EXPLANATION

Code / Syntax	Explanation
class Student { constructor(...) }	Defines ES6 object blueprint for student records
document.getElementById(...)	Retrieves form input and container DOM elements
document.createElement('div')	Creates dynamic card wrapper and item nodes
profileCard.appendChild(...)	Mounts generated profile details into DOM tree
resetForm()	Clears form fields after successful card creation

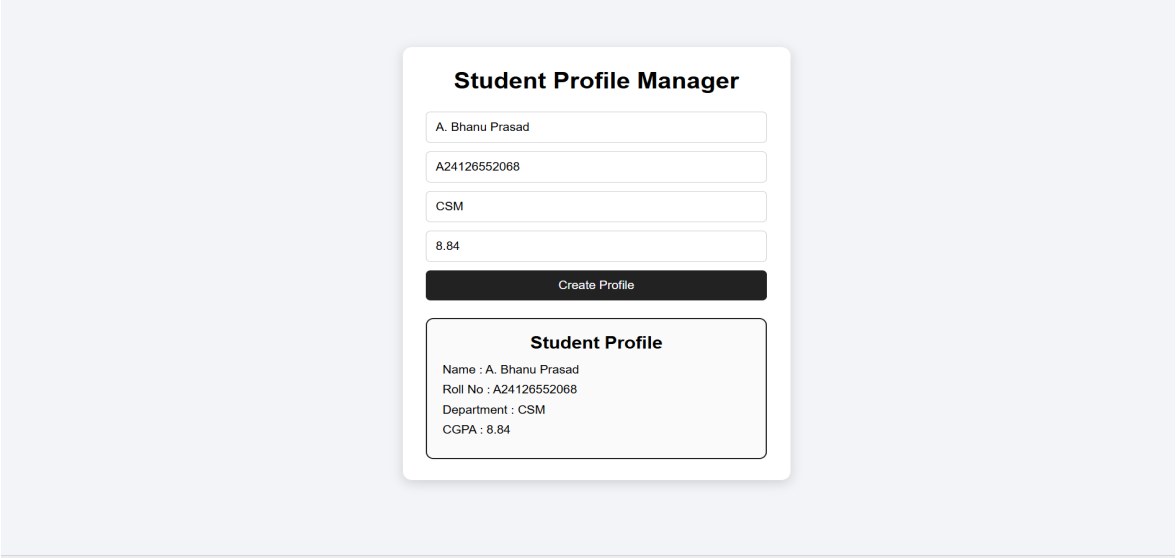
8. TEST CASES AND EXECUTION DATA

The experiment was executed with the following test cases to verify functionality:

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Form Validation	Click Create with empty inputs	Validation prompt to fill all fields	**Pass**
TC-02	Profile Creation	Enter valid student details	Profile card created and rendered	**Pass**
TC-03	Multiple Profiles	Add second student record	New card appended to container	**Pass**

9. OUTPUT

Output 1: Execution Screenshot



10. RESULT

Thus, the experiment for Observation Task 2 (Student Profile Manager Web Application Using JavaScript ES6 and DOM Manipulation) was successfully executed and verified.